

SOFTWARE ENGINEERING PRESENTATION

Multimedia Analyzer for audio, video, alert detection, and structured review

A presentation structured around the implemented system: interface design, processing flow, architectural decisions, implementation evidence, and selected code from the backend and frontend.

ROUTES

11

Auth, upload, watchlist, asset retrieval, processing status, and review-oriented flows.

BACKEND MODULES

19

FastAPI, models, services, tasks, database setup, and routers.

ASSIGNMENT ARTIFACTS

16

Requirements, UML, UI spec, testing strategy, report files, and diagrams.

CORE LAYERS

4

Frontend, API, worker, and persistence working together as one analyst workflow.

INPUTS

Audio and video uploads enter the system through a single dashboard flow.

CORE PIPELINE

Queueing, audio extraction, transcription, entity detection, and alert matching.

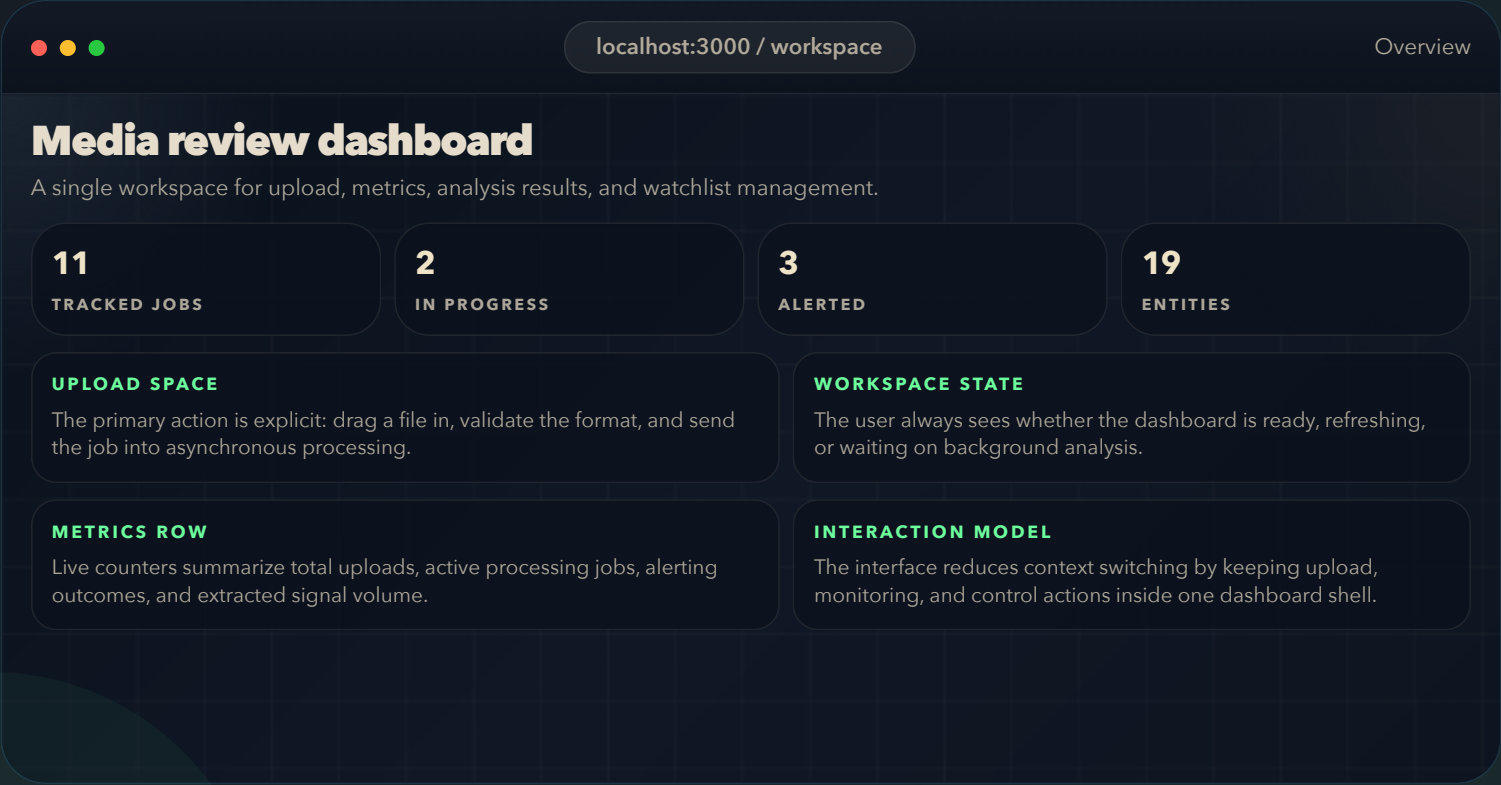
OUTPUTS

Transcript preview, extracted entities, alert state, and persistent analysis history.

DASHBOARD SURFACE

The main workspace presents the whole system in one controlled interface

This screen is where the user enters the system, uploads media, checks job counts, and understands whether the platform is idle, processing, or ready for review.



What this UI is proving

The frontend is not a single upload form. It is a stateful review workspace connected to authentication, media submission, background jobs, and persistent history.

Authentication: the user enters through sign in or registration before any protected action is available.

Upload: the file moves from client validation to API submission and then into the queue.

Monitoring: the workspace immediately reflects active jobs and later refreshes into final review states.

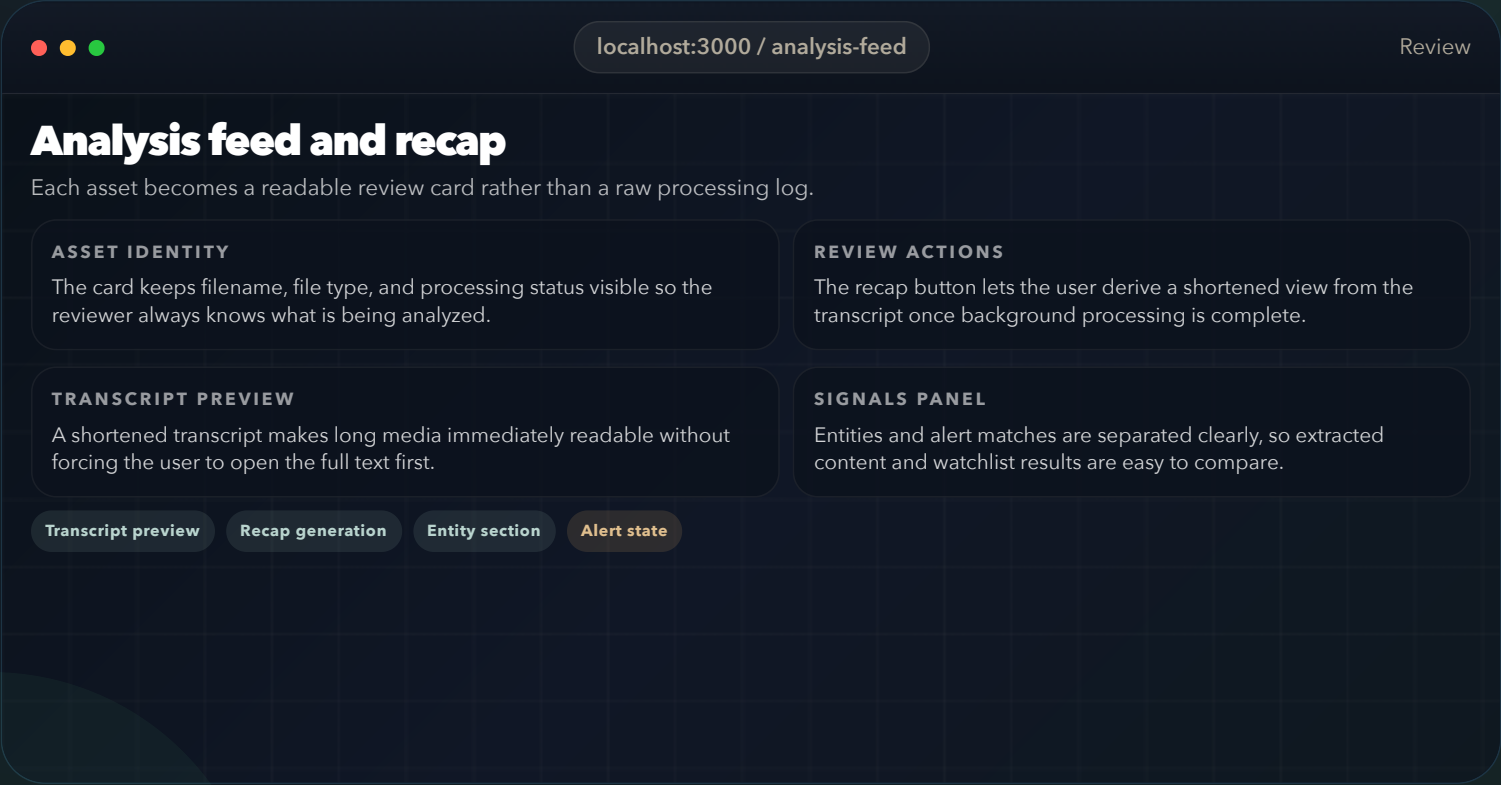
How to explain it

Start with this slide when presenting. It gives the audience the system boundary first: where the user begins, what can be controlled, and where analysis results will appear later.

REVIEW LAYER

The analysis feed converts processing output into readable evidence

The value of the project appears here: raw background processing is transformed into transcript previews, recap text, entity sections, alert matches, and visible status for each uploaded asset.



How the feed is produced

The asset card is the end result of several backend stages: upload, queue creation, FFmpeg audio handling when needed, transcription, entity extraction, alert comparison, and database update.

Transcript: generated by the processing pipeline and stored as the main readable output.

Entities: extracted from transcript content and rendered as structured analyst signals.

Alerts: created by comparing extracted results against the user-defined watchlist.

Why this matters academically

This slide demonstrates that the project is not limited to storage or upload. It reaches a genuine review stage where technical processing becomes visible product value.

USER-CONTROLLED SIGNALS

The watchlist layer defines which entities should trigger attention

The project is not only reactive. The watchlist gives the user a way to shape the system by declaring which names, organizations, or concepts matter before results are reviewed.

localhost:3000 / watchlist-controls

Signals

Watchlist-driven review

The analyst defines the entities that matter before or after upload.

WATCHLIST EDITOR

Comma-separated names are normalized into reusable tags that shape alert detection across the feed.

QUICK STEPS

Sign in, upload file, wait for processing, then review transcript and signals in one continuous flow.

NORMALIZED LIST

The UI shows the cleaned entity list back to the user, which improves traceability and reduces ambiguity before alert evaluation begins.

SYSTEM MESSAGE

Status feedback confirms whether the watchlist was saved successfully and whether the workspace is synchronized with the backend.

OpenAI

Microsoft

TEDx

Innovation

AWS

Control logic behind the UI

The watchlist is part of the functional system, not a decorative preference. It directly influences whether transcript content is escalated as an alert in the review feed.

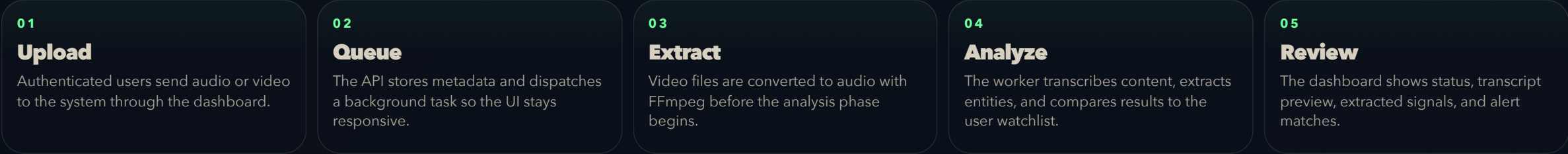
How the alert path works

- Step 1:** the user saves a normalized watchlist through the dashboard.
- Step 2:** the backend persists those entities with the authenticated user profile.
- Step 3:** the worker compares extracted transcript signals against that stored list.
- Step 4:** the asset status becomes *alerted* when relevant matches are found.

MEDIA PIPELINE

From upload to searchable output and entity alerts

The processing chain turns raw media into structured review data: transcript text, extracted entities, watchlist hits, and a final status that the dashboard can present immediately.



Output that matters to the user

Transcript preview Entity labels Alert matches Status badges History feed

The product becomes most valuable at the review stage, where the transcript, extracted names, watchlist hits, and asset status are combined into a readable analyst workflow.

Input: audio / video Output: transcript Output: entities Output: alert status

- Transcript makes spoken content searchable and readable.
- Entities highlight relevant organizations, people, or locations.
- Alert state shows whether the watchlist matched important signals.

API RESPONSIBILITY

The API authenticates the user, validates the upload, stores metadata, and creates the background job.

WORKER RESPONSIBILITY

The worker performs FFmpeg conversion when required, runs transcription, extracts entities, and evaluates alert matches.

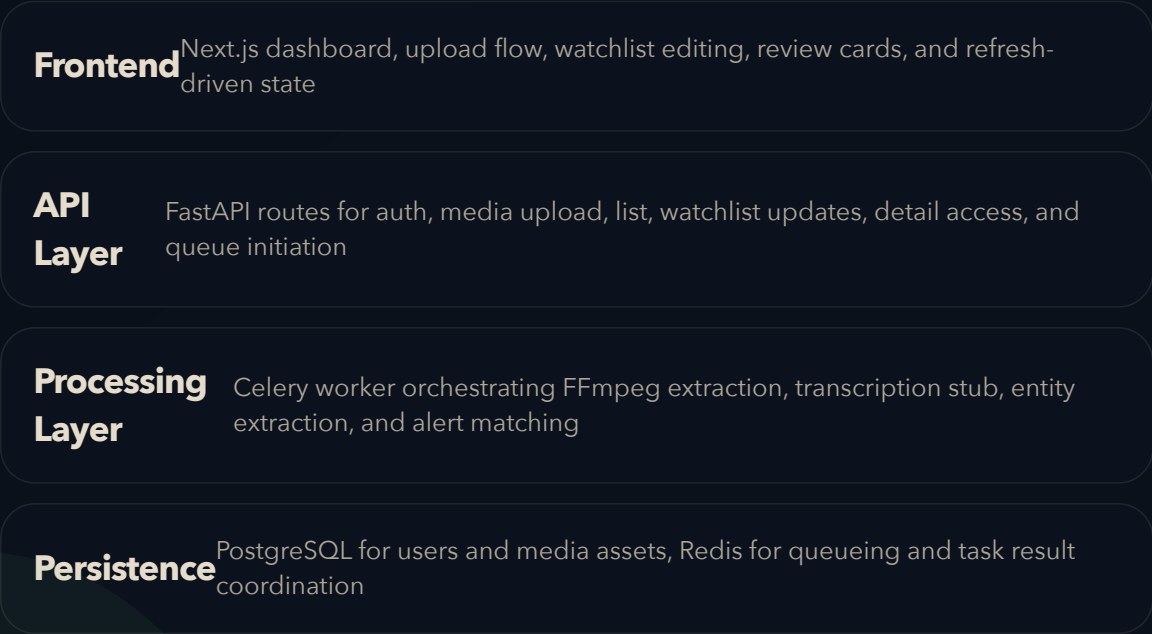
FRONTEND RESPONSIBILITY

The dashboard polls for updated states and turns backend outputs into a readable sequence of cards, counts, and review actions.

IMPLEMENTATION STRUCTURE

Four layers supporting one coherent system

The architecture is simple enough to explain clearly in a coursework setting, but separated enough to justify API design, asynchronous processing, persistence, and frontend state management as distinct concerns.



11 routes

Enough surface area to show a real application API instead of a single demo endpoint or one-page prototype.

19 backend modules

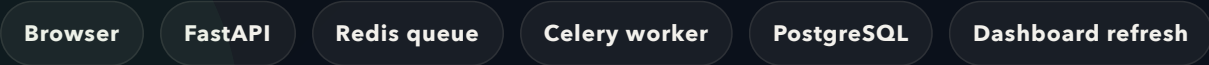
Models, routers, services, tasks, schemas, and configuration are separated cleanly for explanation and maintenance.

6 TSX screens/components

The UI is decomposed into reusable dashboard parts rather than kept in one large file, which improves maintainability and presentation clarity.

Docker-ready stack

The project is packaged for repeatable startup with API, worker, DB, Redis, and frontend, which supports demonstration and deployment consistency.



ASSIGNMENT DELIVERABLES

The project is supported by a full software engineering artifact set

The documentation layer is meant to prove that implementation and specification match: requirements, UML views, UI design, testing strategy, and report packaging all point back to the same product.

Included artifact groups

- Requirements analysis
- Prioritized requirements
- Use case diagram
- Conceptual class model
- State diagrams
- Sequence diagram
- Activity diagrams
- UI design spec
- Testing strategy
- AI prototype prompt
- Report files

Artifact footprint



In a PSE context, implementation alone is not enough. The project also needs to demonstrate structure, reasoning, traceability, and implementation evidence.

- Requirements -> Use cases
- Use cases -> UML
- UML -> UI
- UI -> Tests

Specification

What the system should do and which actors are involved.

Design

How the system is structured and how data moves through it.

Prototype

Which screens and flows are already visible in the working product.

Verification

How tests and reports support the implementation claims.

ENGINEERING FOOTPRINT

A compact project, but broad enough to demonstrate real system design

The codebase is not oversized, yet it covers enough concerns to justify discussion of architecture, asynchronous work, UI state, persistence, and engineering tradeoffs in the presentation.



What the numbers say

Substantial user interface

The frontend includes upload handling, filtered feed review, status synchronization, watchlist state, and user-facing processing feedback.

Beyond simple CRUD

Background processing, file conversion, queue orchestration, and signal extraction move it beyond a simple database app or static dashboard.

Academic suitability

The codebase is small enough to explain clearly and broad enough to justify architecture, UML, testing, and estimation discussion in an academic setting.

SELECTED CODE SEGMENT

Background task orchestration in the media pipeline

app/tasks/media_tasks.py

Celery task

```
@celery_app.task(name="media.process")
def process_media(asset_id: int):
    db = SessionLocal()
    try:
        asset = db.query(MediaAsset).filter(
            MediaAsset.id == asset_id
        ).first()

        user = db.query(User).filter(
            User.id == asset.owner_id
        ).first()

        asset.status = "processing"
        db.add(asset)
        db.commit()

        if asset.media_type == "video":
            pipeline.extract_audio(asset.stored_path, audio_path)
```

Why this code matters

It proves the project uses a real background-processing flow instead of doing everything synchronously in one request, which is essential for larger media files.

Engineering value

Status changes, media-type branching, persistence updates, and failure handling are all visible in one coherent task function tied directly to user-visible results.

SELECTED CODE SEGMENT

Feed card UI for transcript review, entities, and alert visibility

frontend/components/dashboard/FeedPanel.tsx

Asset card

```
function AssetCard({ asset, onDelete, onGenerateRecap }) {
  const canGenerateRecap =
    Boolean(asset.transcript) &&
    (asset.status === "completed" || asset.status === "alerted");

  return (
    <article className={`rounded-[1.7rem] border p-5 ...`} >
      <h3>{asset.original_filename}</h3>
      <span>{asset.media_type}</span>
      <span>.{asset.extension}</span>

      <button onClick={() => onDelete(asset.id)}>Delete</button>
      {asset.alert_count > 0 ? <span>{asset.alert_count} alerts</span>
      : null}
      <span>{asset.status}</span>

      <p>{getAssetSummary(asset)}</p>
      <p>{getStatusDetail(asset.status)}</p>
    </article>
  );
}
```

Why this excerpt matters

It shows that the interface is not decorative only. The feed card is where processing results become readable product value for the analyst.

What the viewer immediately understands

Filename, type, alert state, transcript preview, summary, and recap actions all live in one review component with strong visual hierarchy.

CLOSING SLIDE

Multimedia Analyzer combines system design, UI clarity, and documentable engineering work

Product perspective

The project solves a clear workflow: upload media, process it in the background, detect relevant entities, and present useful results inside a structured analyst dashboard for efficient review.

Engineering perspective

The stack shows real separation of concerns: frontend, API, worker, queue, database, and media conversion logic are all visible and explainable within the repository.

PSE perspective

Requirements, UML, UI specification, testing strategy, report artifacts, charts, screens, and code evidence make the submission coherent and defensible across the full presentation.

Screens from the project

Architecture view

Charts and counts

Two code-focus slides

PSE artifact evidence

This deck is designed to maintain a premium visual standard while still matching the academic expectations of a software engineering submission.